CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

Practical File

On

PYTHON PROGRAMMING Lab

Subject Code: 26CAH-606

Submitted By:

Name: Sahil Sharma

UID: 26MCI10257

Class: MAM-4

Section: B

Submitted To:

Name: Mr. Sanjeev Kumar

Designation: Assistant Professor

Ecode: E20624

Master of Computer Applications (MCA)
University Institute of Computing
Chandigarh University,
Gharuan, Mohali, Punjab, India-140413

UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

INDEX

	Exp No.	Exp Name	Exp Date	Conduct (12 marks)	Worksheet (8 marks)	Viva (10 marks)	Total (30 marks)	Sign

UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Worksheet 2 (A)

Student Name: Sahil Sharma **UID:** 26MCI10257

Branch: MCA (AIML) **Section/Group:** 26MAM-4B

Semester: 1st **Date of Performance:** 18/08/2026

Subject: Python Programming (LAB) **Subject Code:** 26CAH-606

DATE: 18th August, 2026

AIM 1:

WAP to perform list operation-creating, accessing, slicing, modifying, adding, removing, sorting, reversing, finding length, and looping through lists.

OBJECTIVE:

The objective of this experiment is to understand and implement various list operations in Python. The experiment aims to learn how to create, access, slice, modify, add, remove, sort, and reverse elements in a list. It also includes finding the length of a list and using loops to traverse and display its elements.

THEORY:

A list in Python is an ordered and mutable collection of elements. Lists can store multiple values of different data types and are created using square brackets []. Python provides various built-in operations and methods to manipulate lists. Elements can be accessed using their index, and a specific portion of a list can be obtained using slicing. Since lists are mutable, their elements can be modified after creation.
Elements can be added to a list using methods such as append(), insert(), and extend(), while elements can be removed using remove(), pop(), and del. Lists can be arranged in ascending or descending order using the sort() method and can be reversed using the reverse() method. The len() function is used to find the number of elements present in a list.
A for or while loop can be used to traverse and display each element of the list. These operations provide an efficient way to store, access, and manipulate collections of data in Python.

CODE :

```
# Creating a list
my_list = [10, 20, 30, 40, 50]

# Accessing elements
print(my_list[0])
print(my_list[-1])

# Slicing
print(my_list[1:4])

# Modifying elements
my_list[0] = 100
print(my_list)

# Adding elements
my_list.append(60)
print(my_list)

# Removing elements
my_list.remove(30)
print(my_list)

# Sorting
my_list.sort()
print(my_list)

# Reversing
my_list.reverse()
print(my_list)

print(len(my_list))

for item in my_list:
    print(item)
```

OUTPUT:

```
10
50
[20, 30, 40]
[100, 20, 30, 40, 50]
[100, 20, 30, 40, 50, 60]
[100, 20, 40, 50, 60]
[20, 40, 50, 60, 100]
[100, 60, 50, 40, 20]
5
100
60
50
40
20
```

LEARNING OUTCOMES:

1. Understand the concept and structure of lists in Python.
2. Learn how to create and initialize a list.
3. Perform accessing and slicing operations on lists.
4. Modify existing elements in a list.
5. Add and remove elements using built-in list methods.
6. Sort and reverse the elements of a list.
7. Find the length of a list using the len() function.
8. Traverse and display list elements using a for loop.

UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Worksheet 2 (B)

AIM :

WAP to get user id, user name, and user age from user and based on the entered id print the details foe particular user. Hint: use dictionary.

OBJECTIVE:

The objective of this experiment is to understand and implement dictionaries in Python for storing and managing user information. The experiment aims to store user ID, user name, and user age as key-value pairs and retrieve and display the details of a particular user based on the entered user ID.

THEORY:

A dictionary in Python is a collection of data stored in key-value pairs. In this program, a dictionary is used to store user information such as user ID, name, and age. The user ID acts as the key, while the name and age are stored as values in a nested dictionary.
The program allows users to add new users and display details of a particular user by entering their user ID. The 'in' operator is used to check whether the entered ID exists in the dictionary. A while loop provides a menu-driven interface with options to add users, display user details, or exit the program.

CODE :

```
users = {}

n=int(input('enter number of users: '))
for i in range(n):
    user_id = input("Enter user ID: ")
    user_name = input("Enter user name: ")
    user_age = int(input("Enter user age: "))
    users[user_id] = {"name": user_name, "age": user_age}

while True:
    print("1. Add User")
    print("2. Display User Details")
    print("3. Exit")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        user_id = input("Enter user ID: ")
        user_name = input("Enter user name: ")
```

```
    user_age = int(input("Enter user age: "))
    users[user_id] = {"name": user_name, "age": user_age}
elif choice == 2:
    user_id = input("Enter user ID to display details: ")
    if user_id in users:
        print(f"User ID: {user_id}")
        print(f"Name: {users[user_id]['name']}")
        print(f"Age: {users[user_id]['age']}")
    else:
        print("User not found.")
elif choice == 3:
    break
else:
    print("Invalid choice.")
```

OUTPUT:

```
Enter user ID: 101
Enter user name: Raja
Enter user age: 23
Enter user ID: 102
Enter user name: Rohit
Enter user age: 24
Enter user ID: 103
Enter user name: Rashi
Enter user age: 21
1. Add User
2. Display User Details
3. Exit
Enter your choice: 2
Enter user ID to display details: 101
User ID: 101
Name: Raja
Age: 23
1. Add User
2. Display User Details
3. Exit
Enter your choice: 3
```

LEARNING OUTCOMES:

1. Understand the concept and structure of dictionaries in Python.
2. Learn to store user information using key-value pairs.
3. Understand the use of nested dictionaries for storing multiple details.
4. Learn to add new user records to a dictionary.
5. Retrieve and display user details using a user ID.
6. Check whether a particular user ID exists in the dictionary.
7. Use loops and conditional statements to create a menu-driven program.
8. Develop basic skills in managing and accessing data using Python dictionaries.